# Exercicis Vue.js amb TypeScript

> Nivell: Bàsic → Intermedi · Options API

---

## Bloc 1 — Sintaxi bàsica i reactivitat

### Exercici 1 — Targeta de presentació

Crea un component `TarjetaView.vue` que mostri una targeta personal amb les dades:

- Nom, cognoms, edat i professió
- Declara-les totes a `data()` amb TypeScript
- Afegeix un botó que incrementi l'edat en +1 cada cop que es clica

**Conceptes:** `data()`, interpolació `{{ }}`, `@click`

---

### Exercici 2 — Commutador de llum

Crea un component que simuli un interruptor de llum:

- Un botó que alterna entre "Llum encesa 💡 " i "Llum apagada 🌑 "
- El fons del component canvia de groc clar a gris fosc depenent de l'estat
- Usa `:class` o `:style` per al canvi de color

**Conceptes:** `data()` booleà, `@click`, `:class` dinàmic

---

### Exercici 3 — Conversió de temperatura

Crea un formulari amb un `<input>` on l'usuari introdueix una temperatura en Celsius:

- Mostra la conversió a Fahrenheit i a Kelvin en temps real
- Usa `v-model` per capturar el valor de l'input
- Les conversions han de ser propietats `computed`

**Fórmules:** `F = C × 9/5 + 32` · `K = C + 273.15`

**Conceptes:** `v-model.number`, `computed`

---

### Exercici 4 — Llista de la compra

Crea una app de llista de la compra:

- Un `<input>` + botó "Afegir" per afegir productes
- Cada element de la llista té un botó "Eliminar" (✕) al costat
- Si el camp està buit, el botó "Afegir" ha d'estar desactivat (`:disabled`)
- Mostra el total d'elements al final

**Conceptes:** `v-model`, `v-for`, `@click`, `:disabled`, array en `data()`

---

### Exercici 5 — Validació de formulari simple

Crea un formulari de registre amb els camps: nom (obligatori, mínim 3 caràcters), correu electrònic (ha de contenir `@`) i contrasenya (mínim 6 caràcters):

- Mostra missatges d'error sota cada camp quan és invàlid
- El botó "Enviar" només s'habilita si tots els camps són vàlids
- Usa `computed` per calcular si el formulari és vàlid

**Conceptes:** `v-model`, `computed`, `:disabled`, `v-show` o `v-if`

---

## Bloc 2 — Directives i renderització

### Exercici 6 — Semàfor

Crea un component semàfor que cicli entre verd, groc i vermell:

- Un botó "Següent" avança al color següent
- El cercle del semàfor es pinta del color actiu amb `:style`
- Mostra el text del color actiu i quin és el significat ("Passa", "Prepara't", "Para")

**Conceptes:** `:style`, `v-if` / `v-else-if`, array de strings en `data()`

---

### Exercici 7 — Taula de multiplicar

Crea un component que mostri la taula de multiplicar d'un número:

- Un `<input type="number">` per triar el número (1-10)
- Mostra les 10 multiplicacions amb `v-for` dins una `<table>`
- La fila activa (resultat > 50) es ressalta en groc

**Conceptes:** `v-model.number`, `v-for` amb rang, `:class` condicional

---

### Exercici 8 — Filtre de llista

Tens un array d'objectes `{ id, nom, categoria, preu }` amb almenys 8 productes:

- Un `<select>` per filtrar per categoria
- Un `<input>` per buscar per nom
- Mostra els productes filtrats amb `v-for`
- Mostra quants resultats s'han trobat

**Conceptes:** `v-for`, `v-model`, `computed` per al filtre

---

### Exercici 9 — Acordió

Crea una llista de 4 preguntes freqüents (FAQ) amb resposta:

- Cada pregunta es pot expandir/contraure individualment fent clic
- Mostra un icone ▶ o ▼ depenent de si està obert o no
- Defineix una interfície TypeScript `Faq { id: number; pregunta: string; resposta: string; oberta: boolean }`

**Conceptes:** `v-for`, `@click`, `v-show`, interfície TypeScript, `data()` tipat

---

### Exercici 10 — Taulell de notes

Crea una aplicació de notes estil Post-it:

- Un formulari per afegir notes (títol + text)
- Les notes es mostren com a targetes en una graella
- Cada nota té un botó per eliminar-la
- No es pot afegir una nota si el títol és buit
- Defineix la interfície `Nota { id: number; titol: string; text: string }`

**Conceptes:** `v-for`, `v-model`, `:key`, `v-if`, interfície TypeScript

---

## Bloc 3 — Components i comunicació

### Exercici 11 — Component Avatar

Crea un component reutilitzable `AvatarComp.vue` que accepti les props:

- `nom: string` (obligatori)
- `inicials: string` (obligatori, mostrades dins un cercle de color)
- `color: string` (opcional, per defecte `'#42b883'`)
- `mida: 'petit' | 'mitjà' | 'gran'` (opcional, per defecte `'mitjà'`)

Al component pare, renderitza 3 avatars amb dades diferents.

**Conceptes:** `props`, `PropType`, valors per defecte, component fill

---

### Exercici 12 — Component Botó personalitzat

Crea un component `BotoComp.vue` que empaqueti un `<button>`:

- Props: `etiqueta: string`, `variant: 'primari' | 'secundari' | 'perill'`, `desactivat: boolean`
- Cada variant té un color diferent via `:class`
- Emet un event `@clicat` cap al pare quan es clica
- El pare escolta l'event i mostra un missatge de quin botó s'ha premut

**Conceptes:** `props`, `emits`, `$emit`, event custom

---

### Exercici 13 — Comptador pare-fill

Crea un component pare amb un array de 3 comptadors. Cada comptador és un component fill `ComptadorComp.vue` amb:

- Props: `id: number`, `valorInicial: number`, `etiqueta: string`
- Botons `+` i `−` dins el fill
- El fill emet `@actualitzat` amb el nou valor
- El pare mostra la suma total de tots els comptadors

**Conceptes:** `props`, `emits` tipats, comunicació fill → pare, `computed`

---

### Exercici 14 — Formulari amb v-model personalitzat

Crea un component `InputValidat.vue` que funcioni com a input amb validació:

- Accepta `modelValue: string` com a prop
- Emet `update:modelValue` quan l'usuari escriu
- Accepta una prop `regla: (valor: string) => string | null` (retorna missatge d'error o null si és vàlid)
- Al component pare, usa-lo amb `v-model` per a un camp de correu

**Conceptes:** `v-model` personalitzat, `modelValue`, `update:modelValue`, prop de tipus funció

---

### Exercici 15 — Targeta amb slots

Crea un component `TarjetaComp.vue` amb tres slots:

- `#capçalera` — per al títol
- slot per defecte — per al contingut
- `#peu` — per a les accions (botons)

Al component pare, instancia la targeta dues vegades amb contingut diferent:

1. Una targeta d'informació d'usuari
2. Una targeta d'avís d'error

**Conceptes:** `<slot>`, slots nominals, contingut per defecte del slot

---

## Bloc 4 — Reactivitat avançada

### Exercici 16 — Historial de cerca

Crea un component de cerca amb historial:

- Un `<input>` de cerca
- Un `computed` que filtri una llista de 10 ciutats
- Cada cop que l'usuari fa clic a un resultat, s'afegeix a un historial
- L'historial no pot tenir duplicats

- Un botó "Netejar historial"

**Conceptes:** `v-model`, `computed`, `watch` per detectar canvis a l'input

---

## Exercici 17 — Desament automàtic

Crea un editor de text senzill:

- Un `<textarea>` amb `v-model`
- Usa `watch` per detectar canvis i simular un desament automàtic (mostra "Desant..." 1 segon i després "Desat ✓")
- Utilitza `deep: false` i el paràmetre `immediate: false`
- Mostra la data i hora del darrer desament

**Conceptes:** `watch`, `immediate`, temporitzadors, `data()` tipat

---

## Exercici 18 — Cistella de la compra

Crea una app de cistella de la compra:

- Array de `Producte { id: number; nom: string; preu: number; quantitat: number }`
- Botons `+` i `−` per modificar la quantitat de cada producte
- `computed` per al total (suma de `preu × quantitat`)
- `computed` per al nombre total d'articles
- Un botó "Buidar cistella" que reseteja totes les quantitats a 0

**Conceptes:** `computed` amb càlcul d'arrays, `v-for`, TypeScript interfaces

---

## Exercici 19 — Rellotge en temps real

Crea un component que mostri l'hora actual actualitzada cada segon:

- Mostra hores, minuts i segons formatats (`HH:MM:SS`)
- Usa `mounted()` per iniciar un `setInterval`
- Usa `unmounted()` per netejar l'interval
- Afegeix un botó "Pausa / Reprèn"

**Conceptes:** `mounted`, `unmounted`, `setInterval`, `clearInterval`, `data()` tipat

---

## Exercici 20 — Provide / Inject — Tema de l'aplicació

Crea una app amb tres nivells de components: `App → Panell → Contingut`:

- `App.vue` proporciona (`provide`) un objecte `tema: { color: string; fosc: boolean }`
- `Panell.vue` és un component intermediari que **no** té accés directe al tema
- `Contingut.vue` injecta (`inject`) el tema i aplica els colors corresponents
- Un botó a `App.vue` canvia entre tema clar i fosc

**Conceptes:** `provide`, `inject`, comunicació entre components sense props directes

---

## Bloc 5 — Router i navegació

### Exercici 21 — Navegació amb paràmetres

Crea una app amb Vue Router amb dues pàgines:

- `LlistaUsuarisView.vue` — mostra una llista de 5 usuaris (`{ id, nom, rol }`)
- `DetallUsuariView.vue` — rep `id` com a paràmetre de ruta i mostra el detall
- Si l'`id` no existeix, mostra un missatge "Usuari no trobat"
- Afegeix un botó "← Tornar" que navega a la llista

**Conceptes:** `useRoute` / `$route.params`, `RouterLink`, `RouterView`, rutes dinàmiques

---

### Exercici 22 — Formulari multipàgina (Wizard)

Crea un formulari de registre en 3 passos amb Vue Router:

- Pas 1 (`/registre/pas-1`): Dades personals (nom, cognom)
- Pas 2 (`/registre/pas-2`): Dades de contacte (correu, telèfon)
- Pas 3 (`/registre/pas-3`): Resum i confirmació
- Les dades de cada pas han de persistir en navegar entre passos (guarda-les al component pare)
- Una barra de progrés indica en quin pas es troba l'usuari

**Conceptes:** rutes niades o seqüencials, `$router.push`, comunicació entre vistes

---

## Bloc 6 — Exercicis integrats

### Exercici 23 — Gestor de tasques (To-Do)

Crea una app To-Do completa:

- Interfície `Tasca { id: number; text: string; completada: boolean; prioritat: 'alta' | 'normal' | 'baixa' }`
- Afegir noves tasques amb prioritat seleccionable
- Marcar tasques com a completades
- Filtrar per estat: "Totes", "Pendent", "Completada"
- Filtrar per prioritat
- Eliminar tasques completades d'una vegada
- Mostra el recompte de tasques pendents

**Conceptes:** interfícies TypeScript, `computed`, `v-model`, `v-for`, `@click`, `v-if`

---

### Exercici 24 — Joc de memòria (Parelles)

Crea un joc de parelles de cartes:

- Genera un tauler de 4×4 amb 8 parelles de cartes (usa emojis)
- L'usuari pot girar 2 cartes alhora; si coincideixen, resten obertes
- Si no coincideixen, es tanquen de nou
- Mostra el nombre d'intents i de parelles trobades
- Botó "Nova partida" que reinicia el joc

**Conceptes:** TypeScript interfaces, `v-for`, `@click`, `:class`, `methods`, lògica de joc

---

### Exercici 25 — Dashboard d'estadístiques

Crea un dashboard amb dades fictícies de vendes:

- Array de `Venda { id: number; producte: string; import: number; mes: number; categoria: string }`
- `computed` per a: total facturat, venda màxima, venda mínima, mitjana
- Filtre per mes i per categoria (ambdós actius simultàniament)
- Una taula amb les vendes filtrades (ordenades per import, de major a menor)
- Un resum que es recalcula automàticament quan canvien els filtres

**Conceptes:** `computed` encadenat, `v-model`, `v-for`, TypeScript interfaces, mètodes d'array (`.filter`, `.sort`, `.reduce`)

---

### Exercici 26 — Components dinàmics amb KeepAlive

Crea un component que mostri tres pestanyes intercanviables usant `<component :is>`:

- Les pestanyes han de ser tres components separats: `PestanyaInfo`, `PestanyaEstadistiques` (amb comptador intern), `PestanyaConfiguracio` (amb formulari intern)
- Un computed `componentActiu` retorna la definició del component actiu
- Un `<checkbox>` activa o desactiva `<KeepAlive>`
- Demostra la diferència: sense KeepAlive l'estat es perd en canviar de pestanya; amb KeepAlive es conserva
- Mostra el codi d'exemple de com usar `<component :is>` i `<KeepAlive>`

**Conceptes:** `<component :is>`, `<KeepAlive>`, subcomponents inline (`defineComponent`), `computed` que retorna una definició de component, `:key` per forçar re-renderitzat

---

## Notes per als exercicis

- **TypeScript**: usa sempre `lang="ts"` al bloc `<script>`
- **Options API**: declara les dades a `data()`, la lògica a `methods`, les derivades a `computed` i els observadors a `watch`
- **No uses** `<script setup>` ni Composition API (tret que s'indiqui explícitament)
- **Crea cada exercici** en un projecte independent o com a vista dins un projecte multi-pàgina amb Vue Router
- Per crear el projecte: `npm create vue@latest nom-projecte -- --ts --router`